

使用 XMLHttpRequest 对象向服务器发送HTTP请求

[AJAX简介](#)

[示例代码](#)

[通过 XMLHttpRequest 对象实现 AJAX 请求的步骤](#)

[设置状态监听函数](#)

[从服务器获取数据](#)

[使用 addEventListener 函数添加事件监听器](#)

AJAX简介

- AJAX是异步的JavaScript和XML（Asynchronous JavaScript And XML）。浏览器最早就是使用 `XMLHttpRequest` 对象实现 AJAX与服务器通信。
- 早期AJAX使用XML格式的数据与服务器进行通信，现代浏览器使用JSON格式的数据与服务器进行通信。
- AJAX最主要的两个特性：
 - a. 在不重新加载页面的情况下发送请求给服务器。
 - b. 接受并使用从服务器发来的数据。

示例代码

```
1  // 1.创建 XMLHttpRequest 对象
2  var xhr = new XMLHttpRequest();
3  // 2.设置状态监听函数
4  // 当 xhr 对象的 readyState 状态发生变化时，就会触发回调函数。
5  xhr.onreadystatechange = function () {
6      if (xhr.readyState !== 4) return;
7      if (xhr.status === 200) {
8          // 成功：从服务器获取数据，通过responseText属性获取响应的文本。
9          console.log(xhr.responseText)
10         // 后续处理
11     } else {
12         // 失败，根据响应码判断失败原因
13         new Error(xhr.statusText)
14     }
15 };
16 // 3.规定请求的类型、URL 以及是否异步处理请求
17 xhr.open("GET", url, true);
18 // 4.将请求发送到服务器
19 xhr.send();
```

通过 XMLHttpRequest 对象实现 AJAX 请求的步骤

1. 创建 XMLHttpRequest 对象
2. 设置状态监听函数
3. 规定请求的类型、URL 以及是否异步处理请求
4. 将请求发送到服务器
5. 如果需要像 HTML 表单那样提交数据，请使用 `setRequestHeader()` 来添加 HTTP 头。

```
1  xhr.open("POST",url,true);
2  xhr.setRequestHeader("Content-type","application/x-www-form-urlencoded");
3  xhr.send("fname=Henry&lname=Ford");
```

设置状态监听函数

- 当HTTP请求通过 `XMLHttpRequest` 对象发送到服务器时，我们需要执行一些基于响应的任务，以便处理服务器发回的响应。

- 每当 `XMLHttpRequest` 对象的 `readyState` 属性发生改变时，就会触发 `readystatechange` 事件。
- `XMLHttpRequest` 对象的 `readyState` 属性保存有 `XMLHttpRequest` 对象的状态信息。
- 下面是 `XMLHttpRequest` 对象的三个重要的属性：
 - `onreadystatechange` – 用于存储回调函数，每当 `readyState` 属性发生改变时，就会调用该函数。
 - `readyState` – 存有 `XMLHttpRequest` 对象的状态。请求状态从 0 到 4 发生变化。
 - 0: 请求未初始化
 - 1: 服务器连接已建立
 - 2: 请求已接收
 - 3: 请求处理中
 - 4: 请求已完成，且响应已就绪。
 - `status` – HTTP 响应码
 - 200: "OK"
 - 404: 未找到页面
 - 其他的响应码
- 每当 `readyState` 发生变化时就会调用 `onreadystatechange` 函数。
- 当 `readyState` 等于 4 且状态为 200 时，表示响应已就绪。

```
1  // 设置状态监听函数
2  xhr.onreadystatechange = function () { // 状态发生变化时，触发回调函数
3      if (xhr.readyState !== 4) return;
4      if (xhr.status === 200) {
5          // 成功：从服务器获取数据，通过responseText获取响应的文本
6          console.log(xhr.responseText)
7          // 后续处理
8      } else {
9          // 失败，根据响应码判断失败原因
10         new Error(xhr.statusText)
11     }
12 };
```

从服务器获取数据

- 当 `XMLHttpRequest` 对象的 `readyState` 等于 4 且状态为 200 时，表示响应已就绪，这

个时候就可以用服务器返回的数据做后续的处理了。

- 可以通过 `XMLHttpRequest` 对象的 `responseText` 属性访问这些数据。
 - `responseText` – 服务器响应以文本字符的形式返回。

使用 `addEventListener` 函数添加事件监听器

- 除了通过 `onreadystatechange` 属性向 `XMLHttpRequest` 对象注册事件监听器，也可以使用 `addEventListener` 函数完成同样的任务。

```
1 xhr.addEventListener("readystatechange", () => {
2     if (xhr.readyState !== 4) return;
3     if (xhr.status === 200) {
4         // 成功：从服务器获取数据，通过responseText属性获取响应的文本。
5         console.log(xhr.responseText)
6         // 后续处理
7     } else {
8         // 失败，根据响应码判断失败原因
9         new Error(xhr.statusText)
10    }
11 });
```